Hands-on: Accelerating k-NN with Rcpp

Overview

In this 30 min session you will:

1. Download and inspect the C++ implementation for k-NN.
2. Compile and load the Rcpp code.
3. Run benchmarks comparing pure-R vs Rcpp predictions.
4. Analyze how sample size and dimensionality affect performance.

Setup

Download the C++ and helper scripts into your working directory:

```
curl -O https://raw.githubusercontent.com/mmadoliat/KNNreg/refs/heads/main/R/knn_pred.R
curl -O https://raw.githubusercontent.com/mmadoliat/KNNreg/refs/heads/main/src/knn_pred.cpp
curl -O https://raw.githubusercontent.com/mmadoliat/KNNreg/refs/heads/main/inst/examples/KNN
```

Open the files in your editor to review the code:

- **knn_pred.R** contains the `knn_pred()` function (R).
- **knn_pred.cpp** contains the `knn_pred_cpp()` function (Rcpp).
- **KNNreg-Rcpp.R** sources both R and C++ implementations and runs `microbenchmark()`.

---

1. Compile the C++ code

In an R console or RStudio, run:

```
Rcpp::sourceCpp("knn_pred.cpp")
```

If successful, you should see `knn_pred_cpp` available:

```
ls("package:base") # confirm knn_pred_cpp is loaded
# [1] "knn_pred_cpp"
```

---

2. Inspect the runner script

Open **KNNreg-Rcpp.R**, which contains:

```
library(microbenchmark)

# compile Rcpp code once at startup
Rcpp::sourceCpp("knn_pred.cpp")

# load R functions
source("knn_pred.R")

set.seed(42)
n <- 1000; p <- 5; m <- 200; k <- 10
Xtrain <- matrix(rnorm(n*p), n, p)
Ytrain <- rnorm(n)
Xtest  <- matrix(rnorm(m*p), m, p)

# warm up & check correctness
pred_R <- knn_pred(Xtrain, Ytrain, Xtest, k, method="R")
pred_C <- knn_pred(Xtrain, Ytrain, Xtest, k, method="cpp")
stopifnot(max(abs(pred_R$preds - pred_C$preds)) < 1e-12)

# benchmark
mb <- microbenchmark(
  pure_R = knn_pred(Xtrain, Ytrain, Xtest, k, method="R"),
  Rcpp   = knn_pred(Xtrain, Ytrain, Xtest, k, method="cpp"),
  times  = 20
```

```
)
print(mb)
```

Try running this script:

```
source("inst/examples/KNNreg-Rcpp.R")
```

---

3. Vary parameters

Modify **KNNreg-Rcpp.R** or re-run interactively to examine different settings:

- Increase **n** (training size) from 1000 to 5000 or 10000.
- Increase **p** (dimensions) from 5 to 20 or 50.
- Observe how the Rcpp version scales relative to pure-R.

Focus on how the Rcpp implementation stays much faster as complexity grows.

---

4. Discussion

- Where does Rcpp help most?
- Are there settings where pure R is sufficient?
- How might you further optimize (e.g., using STL partial_sort)?

Next steps

- Try integrating this into a Shiny app.
- Explore parallel Rcpp implementations (OpenMP).
- Consider other statistical routines with nested loops.